Lab 1a Working Docs

Initial Instructions

```
EC2 Web App → RDS (MySQL) “Notes” App
Goal

Deploy a simple web app on an EC2 instance that can:
  Insert a note into RDS MySQL
  List notes from the database

Requirements
  RDS MySQL instance in a private subnet (or publicly accessi
ble if you want ultra-simple)
  EC2 instance running a Python Flask app
  Security groups allowing EC2 → RDS on port 3306
  Credentials stored in AWS Secrets Manager (recommended) or
plain env vars (simpler)

Part 1 — Create RDS MySQL
  Option A (recommended, still simple): RDS private + EC2 pub
lic
    RDS Console → Create database
    Engine: MySQL
    Template: Free tier (or Dev/Test)
    DB instance identifier: lab-mysql
    Master username: admin
    Password: generate or set (keep it safe)
    Connectivity
    VPC: default (or class VPC)
    Public access: No
    VPC security group: create new sg-rds-lab
    Create DB
```

```
Security group for RDS (sg-rds-lab): This is the key “real-wo
rld” security pattern: allow DB access only from the app serv
er’s SG.
  Inbound
    MySQL/Aurora (TCP 3306) Source = sg-ec2-lab (we’ll create
that next)
Outbound
    default allow-all is fine: Don't touch this or Chewbacca
will touch you.

Part 2 — Launch EC2
  1) EC2 Console → Launch instance
  2) Name: lab-ec2-app
  3) AMI: Amazon Linux 2023
  4) Instance type: t3.micro (or t2.micro)
  5) Key pair: choose/create (only if you want SSH access)
  6) Network: same VPC as RDS
  7) Security group: create sg-ec2-lab

Security group for EC2 (sg-ec2-lab)
  Inbound
    HTTP TCP 80 from 0.0.0.0/0 (so they can test in browser)
    (Optional) SSH TCP 22 from your IP only

  Outbound
    allow-all (default)

Now go back to RDS SG inbound rule
Set Source = sg-ec2-lab for TCP 3306.

Part 3 — Store DB creds (Secrets Manager)
  1) Secrets Manager → Store a new secret
  2) Secret type: Credentials for RDS database
  3) Username/password: admin + your password
  4) Select your RDS instance lab-mysql
  5) Secret name: lab/rds/mysql
```

```
Create an IAM Role for EC2 to read the secret
  1) IAM → Roles → Create role
  2) Trusted entity: EC2
  3) Add permission policy (tightest good enough for lab):
      SecretsManagerReadWrite is too broad (but easy).
      Better: create a small inline policy like below.

Inline policy (recommended)
Replace <REGION>, <ACCOUNT_ID>, and secret name if different:
#Check inline_policy.json in this folder

  4) Attach role to EC2:
      EC2 → Instance → Actions → Security → Modify IAM role →
select your role

Part 4 — Bootstrap the EC2 app (User Data)
In EC2 launch, you can paste this in User data (or run manual
ly after SSH).
Important: Replace SECRET_ID if you used a different name.
#user.data.sh

Part 5 — Test
  1) In RDS console, copy the endpoint (you won't paste it in
to app because Secrets Manager provides host)
  2) Open browser:
      http://<EC2_PUBLIC_IP>/init
      http://<EC2_PUBLIC_IP>/add?note=first_note
      http://<EC2_PUBLIC_IP>/list
  If /init hangs or errors, it's almost always:
    RDS SG inbound not allowing from EC2 SG on 3306
    RDS not in same VPC/subnets routing-wise
    EC2 role missing secretsmanager:GetSecretValue
    Secret doesn't contain host / username / password fields
(fix by storing as "Credentials for RDS database")
```

```
Student Deliverables:
1) Screenshot of:
  RDS SG inbound rule using source = sg-ec2-lab
  EC2 role attached
  /list output showing at least 3 notes

2) Short answers:
  A) Why is DB inbound source restricted to the EC2 security
group?
  B) What port does MySQL use?
  C) Why is Secrets Manager better than storing creds in cod
e/user-data?
```

Part 0 (First thing): Create the VPC (Helga)

1. **Plan your CIDR + AZs first (avoid rework)**
   - VPC CIDR: **10.212.0.0/16** (Helga)
   - Pick at least 2 AZs in **us-east-2** (example: us-east-2a + us-east-2b)
   - Decide up front: **Public subnet for EC2**, **Private subnets for RDS**
2. **Create the VPC**
   - VPC Console → **Your VPCs** → **Create VPC**
   - Name tag: **Helga**
   - IPv4 CIDR block: **10.212.0.0/16**
   - Tenancy: Default
3. **Create subnets (minimum viable lab layout)**
   - VPC Console → **Subnets** → **Create subnet**
   - Create at least:
     - **Public subnet** (for EC2)

    - **Private subnet(s)** (for RDS)
  - Put subnets in the AZs you chose (example: us-east-2a for EC2)
4. **Internet Gateway (required if EC2 is public and using User Data without NAT/endpoints)**
  - VPC Console → **Internet Gateways** → Create IGW → **Attach to Helga**
5. **Route tables (the part that usually breaks bootstrap)**
  - Create / verify a **public route table** associated to the public subnet
    - Add route: `0.0.0.0/0 → Internet Gateway (igw-...)`
  - Private subnets should *not* have a route to the IGW
    - If you need outbound internet from private subnets, use **NAT Gateway** (costs money)
6. **Enable public IPv4 on the public subnet (so EC2 is reachable)**
  - Subnet → **Edit subnet settings** → enable auto-assign public IPv4 (or enable it at EC2 launch)
7. **(Optional but recommended) Turn on VPC DNS features**
  - VPC → Edit VPC settings:
    - Enable DNS resolution
    - Enable DNS hostnames
8. **Sanity check before building anything else**
  - Your EC2 subnet route table shows `0.0.0.0/0 → igw-...`
  - You know which subnets are **public** vs **private**
  - You are staying in the same region (**us-east-2**) for VPC, EC2, RDS, and Secrets

---

Part 1 (Expanded, in order): RDS MySQL setup + network prerequisites

1. **Decide how private your EC2 will be (this affects whether you need a VPC Endpoint)**

- If your EC2 will be in a public subnet with an Internet Gateway route, you can reach AWS APIs over the public internet (no endpoint required).
- If your EC2 will be in a private subnet (no direct internet), you must provide a path to AWS APIs.
  - Option A: NAT Gateway (costs money)
  - Option B (preferred for this lab): **VPC Interface Endpoint for Secrets Manager**

2. **(If needed) Create a VPC Interface Endpoint for Secrets Manager**

   Use this if the EC2 instance is private or cannot reach AWS APIs.

   1. VPC Console → Endpoints → Create endpoint
   2. Service category: **AWS services**
   3. Service name: `com.amazonaws.us-east-2.secretsmanager`
   4. Type: **Interface**
   5. VPC: Helga
   6. Subnets: select the subnet(s) where your EC2 lives (or will live)
   7. Security group for the endpoint:
      - Inbound: HTTPS **TCP 443** from **sg-ec2-lab** (once created)
      - Outbound: default allow-all is fine
   8. Private DNS: **Enable**
   9. Create

   Notes:

   - If you also run `aws sts get-caller-identity` from the instance and STS is blocked, you may additionally need the STS interface endpoint.
   - If you are not using endpoints, make sure your subnet has a route to the internet (IGW) or a NAT.

3. **Create the RDS instance**

   a. RDS → Create database

b. Engine: **MySQL**

c. Template: **Free tier** (or Dev/Test)

d. DB identifier: `lab-mysql`

e. Master username: `admin`

f. Set password (store it temporarily until Secrets Manager is created)

4. **Connectivity (make sure this matches how you built your VPC)**
   - VPC: your lab VPC (Helga)
   - Public access: **No** (private DB)
   - DB subnet group: private subnets (if you created them)
5. **Create the RDS Security Group (sg-rds-lab)**
   - Inbound: *do not* open 3306 to the internet.
   - You will add an inbound rule from the EC2 security group after EC2 exists.
6. **RDS ready check**
   - Wait for status: **Available**
   - Copy the RDS endpoint somewhere safe for reference (you'll later verify it matches the secret `host`).

Part 2 (Expanded, in order): EC2 launch + SG wiring + subnet routing

1. **Launch EC2**

a. EC2 → Launch instance

b. Name: `lab-ec2-app`

c. AMI: **Amazon Linux 2023**

d. Type: `t3.micro` (or `t2.micro`)

e. Network: **same VPC as RDS** (Helga)

f. Subnet: a subnet that can reach the internet if you are using User Data (see step 3)

g. Auto-assign public IP: **Enable** (unless you are using an Elastic IP)

2. **Create EC2 Security Group (sg-ec2-lab)**
   - Inbound:
     - HTTP **TCP 80** from `0.0.0.0/0`
     - SSH **TCP 22** from *your IP only* (optional)
   - Outbound: allow-all (default)
3. **Subnet routing (must be correct or bootstrap fails silently)**
   - Subnet route table must have: `0.0.0.0/0 → Internet Gateway (igw-...)`
   - If this is missing, either:
     - Fix the route table, or
     - Relaunch EC2 in a subnet that already has the IGW route
4. **Wire RDS SG inbound to EC2 SG (the key pattern)**
   - Go back to **sg-rds-lab** inbound rules
   - Add:
     - MySQL/Aurora **TCP 3306**
     - Source: **sg-ec2-lab**
5. **Pre-flight sanity checks before touching Secrets Manager**
   - EC2 is reachable on port 80 (at least the instance is reachable)
   - RDS is Available
   - RDS inbound rule references sg-ec2-lab (not an IP range)

Part 3 (Expanded, in order): Secrets Manager setup + verification

1. **Create the secret the right way (required)**
   a. Secrets Manager → **Store a new secret**
   b. Secret type: **Credentials for RDS database**
   c. Enter master username/password you set on the RDS instance

    d. Select your RDS instance **lab-mysql**
    e. Secret name: **lab/rds/mysql**
2. **Immediately verify the secret fields (this is where people get stuck)**
    a. Open the secret → **Retrieve secret value**
    b. Confirm these keys exist:
        - `host`
        - `username`
        - `password`
        - `port` (usually `3306`)
        - `engine` (mysql)
    c. Confirm `host` exactly matches your RDS endpoint value.
3. **Confirm region alignment (silent failure generator)**
    - RDS, EC2, and the Secret must all be in **us-east-2** for this lab.
4. **Confirm encryption won't block the role**
    a. In the secret details, check **Encryption key**
    b. If it is **aws/secretsmanager**: ok
    c. If it is a **customer managed KMS key**: the EC2 role must have `kms:Decrypt` on that key.
5. **Create IAM role policy that matches the exact secret ARN**
    a. Copy the exact Secret ARN from the console (preferred) *or* run:
        - `aws secretsmanager list-secrets --region us-east-2`
    b. In your inline policy, set Resource to that ARN (your example uses `lab/rds/mysql*`).
    c. Required permission:
        - `secretsmanager:GetSecretValue`
6. **Attach role to EC2 and verify from the instance (fastest proof)**

a. Attach role: EC2 → Instance → Actions → Security → Modify IAM role

b. On EC2, run:

- `aws sts get-caller-identity`
- `aws secretsmanager get-secret-value --secret-id lab/rds/mysql --region us-east-2`

If these commands work, Secrets Manager + IAM are correct before you touch `/init`.

What's Due

Lab 1A Deliverables

VPC name Helga

US-east-2a

CIDR Block 10.212.0.0

1a_inline_policy.json

Example

AWS_EC2_Secrets_Access Policy name

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadSpecificSecret",
      "Effect": "Allow",
      "Action": ["secretsmanager:GetSecretValue"],
      "Resource": "arn:aws:secretsmanager:us-east-2:123456789
012:secret:lab/rds/mysql*"
    }
```

```
    ]
}
```

Part 4

Good question! Let me clarify what's happening here:

Did you use the default secret name?

If you named your secret exactly `lab/rds/mysql` (as shown in Part 3), then the user data script should work as-is — no changes needed.

If the EC2 is already running with user data

Here's the catch: **User Data only runs once** — on the very first boot of the instance. So if you already launched with the wrong SECRET_ID baked in, simply editing the User Data field won't fix it.

Your options:

1. **SSH in and fix it manually**
    - Connect to your EC2 via SSH
    - Find where the app is running (likely `/home/ec2-user/` or similar)
    - Update the SECRET_ID in the script and restart the app
2. **Terminate and relaunch** (cleanest for a lab)
    - Terminate the current EC2
    - Launch a fresh one with the corrected User Data
    - Make sure to attach the IAM role during launch (or immediately after via Actions → Security → Modify IAM role)

Before moving to Part 5 (Test), confirm:

- ☑ ~~IAM role with your Secrets Manager policy is **attached** to the EC2 instance~~
- ☑ ~~SECRET_ID in User Data matches your actual secret name (`lab/rds/mysql`)~~

- ☑ ~~EC2 and RDS are in the same VPC~~
- ☑ ~~RDS security group allows inbound 3306 from the EC2 security group~~

Once those are set, you're ready to test with `/init`, `/add?note=...`, and `/list`! 🚀

Good deduction! Let me answer both:

Part 5

`lab-mysql.XXXXXXXXX.us-east-2.rds.amazonaws.com`

Why copy the endpoint?

Honestly, that step is just for your reference — so you can verify the secret contains the right host. You don't actually paste it anywhere since Secrets Manager stores it for you. A bit confusingly written.

Your hang is likely the secret structure

When you created the secret, did you choose **"Credentials for RDS database"** as the secret type? This is crucial because:

Secret Type	What gets stored
Credentials for RDS database	`username`, `password`, `host`, `port`, `dbname`, `engine` — all auto-populated
Other type of secret (key/value)	Only what you manually enter — probably just `username` and `password`

If the Flask app is trying to pull `host` from the secret and it's not there, it'll hang waiting for a connection it can't make.

1a_user_data.sh

```
#!/bin/bash
dnf update -y
dnf install -y python3-pip
pip3 install flask pymysql boto3

mkdir -p /opt/rdsapp
```

```
cat >/opt/rdsapp/app.py <<'PY'
import json
import os
import boto3
import pymysql
from flask import Flask, request

REGION = os.environ.get("AWS_REGION", "us-east-2")
SECRET_ID = os.environ.get("SECRET_ID", "lab/rds/mysql")

secrets = boto3.client("secretsmanager", region_name=REGION)

def get_db_creds():
    resp = secrets.get_secret_value(SecretId=SECRET_ID)
    s = json.loads(resp["SecretString"])
    # When you use "Credentials for RDS database", AWS usuall
y stores:
    # username, password, host, port, dbname (sometimes)
    return s

def get_conn():
    c = get_db_creds()
    host = c["host"]
    user = c["username"]
    password = c["password"]
    port = int(c.get("port", 3306))
    db = c.get("dbname", "labdb")  # we'll create this if it
doesn't exist
    return pymysql.connect(host=host, user=user, password=pas
sword, port=port, database=db, autocommit=True)

app = Flask(__name__)

@app.route("/")
def home():
    return """
```

```
    <h2>EC2 → RDS Notes App</h2>
    <p>POST /add?note=hello</p>
    <p>GET /list</p>
    """

@app.route("/init")
def init_db():
    c = get_db_creds()
    host = c["host"]
    user = c["username"]
    password = c["password"]
    port = int(c.get("port", 3306))

    # connect without specifying a DB first
    conn = pymysql.connect(host=host, user=user, password=pas
sword, port=port, autocommit=True)
    cur = conn.cursor()
    cur.execute("CREATE DATABASE IF NOT EXISTS labdb;")
    cur.execute("USE labdb;")
    cur.execute("""
        CREATE TABLE IF NOT EXISTS notes (
            id INT AUTO_INCREMENT PRIMARY KEY,
            note VARCHAR(255) NOT NULL
        );
    """)
    cur.close()
    conn.close()
    return "Initialized labdb + notes table."

@app.route("/add", methods=["POST", "GET"])
def add_note():
    note = request.args.get("note", "").strip()
    if not note:
        return "Missing note param. Try: /add?note=hello", 40
0
    conn = get_conn()
```

```
    cur = conn.cursor()
    cur.execute("INSERT INTO notes(note) VALUES(%s);", (not
e,))
    cur.close()
    conn.close()
    return f"Inserted note: {note}"

@app.route("/list")
def list_notes():
    conn = get_conn()
    cur = conn.cursor()
    cur.execute("SELECT id, note FROM notes ORDER BY id DES
C;")
    rows = cur.fetchall()
    cur.close()
    conn.close()
    out = "<h3>Notes</h3><ul>"
    for r in rows:
        out += f"<li>{r[0]}: {r[1]}</li>"
    out += "</ul>"
    return out

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=80, debug=True)
PY

cat >/etc/systemd/system/rdsapp.service <<'SERVICE'
[Unit]
Description=EC2 to RDS Notes App
After=network.target

[Service]
WorkingDirectory=/opt/rdsapp
Environment=SECRET_ID=lab/rds/mysql
ExecStart=/usr/bin/python3 /opt/rdsapp/app.py
Restart=always
```

```
[Install]
WantedBy=multi-user.target
SERVICE

systemctl daemon-reload
systemctl enable rdsapp
systemctl start rdsapp
```

Quick fix

This fix didn’t work Get INIT Running

You can't move an EC2 to a different subnet — it's locked at launch. But you have two fast options:

Option A: Fix the route table (fastest if app started)

1. **VPC Console** → **Subnets** → find the subnet in `us-east-2a` your EC2 is using
2. Click the **Route table** tab → click the route table link
3. **Edit routes** → **Add route:**
   - Destination: `0.0.0.0/0`
   - Target: select **Internet Gateway** → pick the IGW attached to VPC "Helga"
   - (If no IGW exists: **VPC Console** → **Internet Gateways** → **Create** → attach to Helga first)
4. Save
5. Test: `http://<YOUR_ELASTIC_IP>/`

If that works, you're done. If not, go to Option B.

Option B: Terminate and relaunch (clean slate)

Since User Data only runs once, if something broke during bootstrap, you'd need to relaunch anyway:

1. **Terminate** current EC2
2. **Release** the Elastic IP (so you're not charged)
3. **Launch new EC2:**
    - Same VPC "Helga"
    - **Subnet:** pick one that already has an IGW route, OR use the subnet you just fixed in Option A
    - **Auto-assign public IP: Enable**
    - Paste the User Data script
    - Attach the IAM role during launch (under Advanced Details)
4. Test with the new public IP

My recommendation: Try Option A first — it's a 2-minute fix. If the page loads, you're golden. If it times out or errors, go Option B.

Troubleshooting Steps (Restored Reference)

Use this as a *no-error-text* checklist to reconstruct what went wrong.

0) Hard rules (so you don’t waste time)

- **User Data runs only once** (first boot). If bootstrap was wrong, you either fix manually (SSH) or relaunch a fresh instance.
- **You cannot move an EC2 instance to a different subnet** after launch.

1) “Page won’t load at all” (can’t reach EC2 app)

1. **Public IP + inbound rules**
    - Instance has a Public IPv4 address (or Elastic IP).
    - EC2 Security Group inbound has:
        - HTTP **TCP 80** from **0.0.0.0/0**

    - (Optional) SSH **TCP 22** from *your IP only*

2. **Subnet route to the internet (most common)**
    - Subnet route table has: `0.0.0.0/0 → Internet Gateway (igw-...)`
    - If missing:
        - Create IGW
        - Attach IGW to VPC (Helga)
        - Add the route
3. **Instance is actually running the service**
    - On the box (SSH), verify:
        - `systemctl status rdsapp`
        - `sudo journalctl -u rdsapp --no-pager`
        - `curl -sS localhost/` (proves app is up locally)
4. **Flask binding**
    - App must bind to **0.0.0.0** on port **80** (not 127.0.0.1).

2) “/init hangs” (web app reachable, DB setup stalls)

1. **RDS security group source is the EC2 security group**
    - RDS SG inbound:
        - MySQL/Aurora **TCP 3306**
        - Source = **sg-ec2-lab** (not 0.0.0.0/0)
2. **Same VPC / compatible subnets**
    - EC2 and RDS are in the **same VPC**.
    - If RDS is private, EC2 must be able to route to the private subnets (same VPC is necessary but not always sufficient if you created unusual routing).
3. **Secret shape is correct (super common)**
    - Secret must include at least:

    - `host`, `username`, `password` (and usually `port`)
- Easiest way: create secret using **"Credentials for RDS database"** and select the RDS instance.

3) "Secret can't be read" (app runs but can't pull creds)

Follow this strict IAM checklist:

1. **Trust Relationship**
    - IAM Role trust relationship includes `ec2.amazonaws.com`.
2. **Role attached to the EC2 instance**
    - EC2 → Actions → Security → Modify IAM role.
3. **Policy allows GetSecretValue to the exact secret ARN**
    - Prefer the scoped policy example already on this page.
    - Confirm ARN accuracy by copying it from Secrets Manager.
4. **KMS decrypt (only if you used a custom CMK)**
    - If the secret uses a customer-managed KMS key, role needs `kms:Decrypt` for that key.

4) "I updated User Data but nothing changed"

- Expected behavior.
- Fix options:
    1. SSH in and update `/opt/rdsapp/app.py` or the systemd env, then restart:
        - `sudo systemctl restart rdsapp`
    2. Terminate + relaunch with corrected User Data.

5) Clean relaunch checklist (when you're done fighting it)

- Pick the right VPC (Helga) and a subnet that has an IGW route.
- Enable **Auto-assign public IP**.
- Attach IAM role **during launch** (or immediately after).

- Paste User Data once, then leave it alone.
- Confirm RDS SG allows 3306 *from the EC2 SG*.

Notes / lab-specific reminders

- MySQL port is **3306**.
- Endpoint copy step is just to verify your secret has the right `host`; you do not paste it into the app when using Secrets Manager.